

Chuyên đề: Kỹ Thuật Đảo Nhãn Quy Hoạch Động (DP State-Value Inversion)

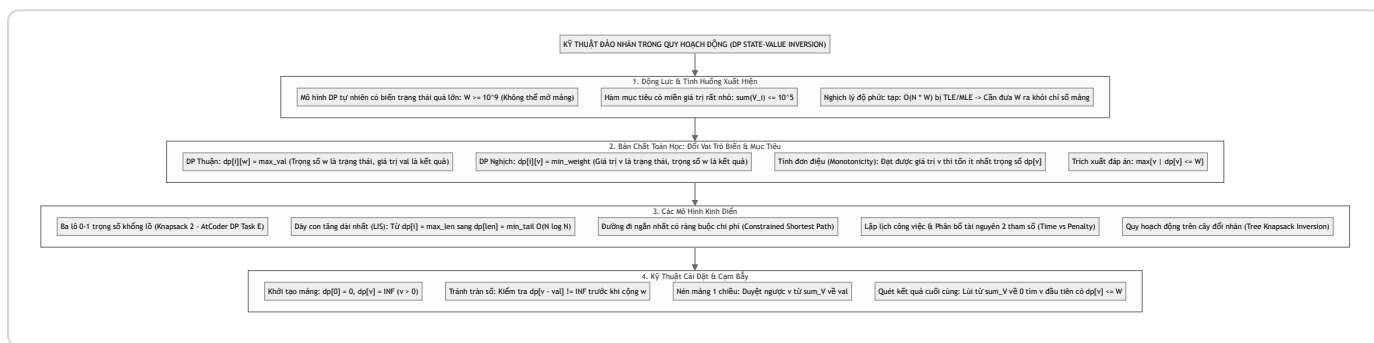
(Swapping DP State and Objective Value: Large Constraints, Dual Formulations & Knapsack 2 Paradigm)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Sơ đồ Tư duy Toàn cảnh (Mindmap)



1. Đặt Vấn Đề: Nghịch Lý Không Gian Trạng Thái Trong DP

Trong quy hoạch động, việc chọn đại lượng nào làm **trạng thái (state index)** và đại lượng nào làm **giá trị lưu trữ (table value)** quyết định trực tiếp đến tính khả thi của thuật toán.

1.1. So sánh hai bài toán Ba lô kinh điển

Xét bài toán chiếc ba lô 0-1 (0-1 Knapsack Problem): Có N đồ vật, đồ vật thứ i có trọng lượng w_i và giá trị v_i . Tìm cách chọn tập đồ vật có tổng trọng lượng không vượt quá W sao cho tổng giá trị là lớn nhất.

Thuộc tính	Bài toán 1 (Knapsack 1 - Task D)	Bài toán 2 (Knapsack 2 - Task E)
Số đồ vật N	$N \leq 100$	$N \leq 100$
Giới hạn trọng lượng W	$W \leq 10^5$ (Nhỏ)	$W \leq 10^9$ (Khổng lồ)
Giá trị mỗi vật v_i	$v_i \leq 10^9$ (Lớn)	$v_i \leq 1000 \implies \sum v_i \leq 10^5$ (Rất nhỏ)

Thuộc tính	Bài toán 1 (Knapsack 1 - Task D)	Bài toán 2 (Knapsack 2 - Task E)
Tiếp cận tự nhiên $dp[w] = \max(v)$	$O(N \cdot W) \approx 10^7$ (AC, 0.03s)	$O(N \cdot W) \approx 10^{11}$ (TLE & MLE)

- Với **Knapsack 1**, biến w nhỏ (10^5) nên ta dùng w làm chỉ số mảng: `dp[w]` = giá trị lớn nhất khi dùng đúng trọng lượng w .
- Với **Knapsack 2**, biến $W \leq 10^9$ khiến ta **không thể khai báo mảng kích thước 10^9** trong bộ nhớ RAM (giới hạn thường là $256 \text{ MB} \approx 6.4 \times 10^7$ số nguyên).

1.2. Ý tưởng cốt lõi: Đảo vai trò (State-Value Inversion)

Quan sát thấy rằng: Trong khi W cực lớn thì **tổng giá trị của tất cả các món đồ lại rất nhỏ**:

$$V_{\max} = \sum_{i=1}^N v_i \leq 100 \times 1000 = 10^5$$

Thay vì hỏi: "Với trọng lượng w , giá trị lớn nhất đạt được là bao nhiêu?", ta **đổi ngược câu hỏi**:

"Để đạt được chính xác tổng giá trị v , trọng lượng nhỏ nhất cần dùng là bao nhiêu?"

Bằng cách hoán đổi vai trò giữa **Trọng lượng (w)** và **Giá trị (v)**:

- Biến w (lớn) được đưa vào **nội dung ô nhớ** (chỉ tốn 1 biến `long long`).
- Biến v (nhỏ) được đưa lên làm **chỉ số mảng** (kích thước mảng chỉ là $V_{\max} \leq 10^5$).
- Độ phức tạp thời gian: $O(N \cdot \sum v_i) \approx 100 \times 10^5 = 10^7$ phép tính $\implies$ Chạy trong **chưa đầy 0.05 giây!**

2. Cơ Sở Toán Học: Tính Đối Ngẫu & Đơn Điều

2.1. Định nghĩa toán học của hai bài toán đối ngẫu

Giả sử ta xét bài toán tối ưu hóa tổng quát với hai thuộc tính của lời giải: chi phí $C(S)$ và lợi ích $B(S)$.

- Bài toán Thuận (Primal Problem):**

$$F(c) = \max_S \{B(S) \mid C(S) \leq c\}$$

Hàm $F(c)$ là hàm đơn điệu tăng không ngặt theo c .

- Bài toán Nghịch / Đảo nhãn (Dual Problem):**

$$G(b) = \min_S \{C(S) \mid B(S) \geq b\}$$

Hàm $G(b)$ là hàm đơn điệu tăng không ngặt theo b .

2.2. Định lý tương đương (Equivalence Theorem)

Với mọi ngưỡng chi phí cho trước $C_{\max}$:

$$\max\{B(S) \mid C(S) \leq C_{\max}\} = \max\{b \mid G(b) \leq C_{\max}\}$$

Chứng minh ngắn gọn:

1. Giả sử $b^* = F(C_{\max})$. Khi đó tồn tại tập S^* sao cho $B(S^*) = b^*$ và $C(S^*) \leq C_{\max}$. Do đó $G(b^*) \leq C(S^*) \leq C_{\max}$.
2. Ngược lại, giả sử tồn tại $b > b^*$ sao cho $G(b) \leq C_{\max}$. Khi đó tồn tại S với $B(S) \geq b > b^*$ và $C(S) \leq C_{\max}$, điều này mâu thuẫn với giả thiết b^* là giá trị lớn nhất có thể đạt được với chi phí $\leq C_{\max}$.
3. Vậy hai bài toán là tương đương nhau. Ta hoàn toàn có thể tính bảng $G(b)$ rồi tìm b lớn nhất thỏa $G(b) \leq C_{\max}$.

3. Các Mô Hình Kinh Điển

3.1. Mô hình 1: Ba lô 0-1 trọng số khổng lồ (Large Weight Knapsack)

Định nghĩa trạng thái:

- Gọi $dp[v]$ là **trọng lượng nhỏ nhất** để đạt được tổng giá trị v .

Khởi tạo:

- $dp[0] = 0$ (Để đạt giá trị 0, cần 0 trọng lượng).
- $dp[v] = \text{INF}$ với mọi $v > 0$ (Ban đầu chưa thể đạt được giá trị v).

Công thức chuyển trạng thái (khi xét đồ vật thứ i có trọng lượng w_i , giá trị v_i):

Duyệt ngược v từ $\sum v$ về v_i (để tránh dùng lại cùng một đồ vật nhiều lần):

$$dp[v] = \min(dp[v], dp[v - v_i] + w_i)$$

Trích xuất kết quả:

Duyệt v từ $\sum v$ lùi về 0, giá trị v đầu tiên thỏa mãn $dp[v] \leq W$ chính là đáp án:

$$\text{Ans} = \max\{v \in [0, \sum v_i] \mid dp[v] \leq W\}$$

3.2. Mô hình 2: Dãy con tăng dài nhất (LIS - Longest Increasing Subsequence)

Thuật toán LIS kinh điển $O(N \log N)$ thực chất là một ứng dụng xuất sắc của **Kỹ thuật Đảo nhãn**:

Cách tiếp cận tự nhiên $O(N^2)$:

- Trạng thái: $dp[i]$ = độ dài dãy con tăng dài nhất **kết thúc tại vị trí i** .
- Chuyển: $dp[i] = 1 + \max_{j < i, A_j < A_i} dp[j]$.
- Số trạng thái N , chuyển mỗi trạng thái tốn $O(N) \implies O(N^2)$.

Cách tiếp cận Đào nhĩn $O(N \log N)$:

- Đổi vai trò giữa **Độ dài (len)** và **Giá trị phần tử kết thúc (A_i)**:
- Trạng thái: $tail[k]$ = **phần tử kết thúc nhỏ nhất** của một dãy con tăng có độ dài đúng bằng k .
- **Tính chất quan trọng**: Mảng $tail$ luôn tăng ngặt theo k ($tail[1] < tail[2] < \dots < tail[len]$).
- Khi xét phần tử A_i : Thay vì duyệt qua mọi độ dài, ta dùng **chặt nhị phân** (`std::lower_bound`) trên mảng $tail$ để tìm vị trí k thích hợp nhất trong $O(\log N)$.
- Cập nhật: $tail[k] = A_i$.
- Độ phức tạp giảm ngoạn mục từ $O(N^2)$ xuống $O(N \log N)$!

3.3. Mô hình 3: Đường đi ngắn nhất có ràng buộc (Constrained Shortest Path)

Xét đồ thị có hướng $G = (V, E)$. Mỗi cạnh (u, v) có hai tham số:

- Độ dài $d(u, v) \in [1, 10^9]$ (Lớn).
- Chi phí $c(u, v) \in [1, 100]$ (Nhỏ).
- Cần tìm đường đi ngắn nhất từ S đến T sao cho **tổng chi phí không vượt quá $C_{\max}$** ($C_{\max} \leq 10^4$).

Áp dụng Đào nhĩn:

- Nếu gọi $dp[u][dist]$ = chi phí nhỏ nhất: $dist$ lên tới $10^9 \implies$ Không thể mở mảng.
- Đào nhĩn: Gọi $dp[u][cost]$ = **độ dài nhỏ nhất** để đi từ S đến u với tổng chi phí đúng bằng $cost$.
- Số trạng thái: $|V| \times C_{\max} \leq 1000 \times 10^4 = 10^7$.
- Chuyển trạng thái bằng thuật toán Dijkstra mở rộng hoặc DP trên đồ thị theo chi phí tăng dần:

$$dp[v][cost + c(u, v)] = \min(dp[v][cost + c(u, v)], dp[u][cost] + d(u, v))$$

- Đáp án: $\min_{c \leq C_{\max}} dp[T][c]$.

4. Trace Chi Tiết Bước Chuyển Trạng Thái (Step-by-Step Trace)

Xét bài toán Ba lô đào nhĩn: $W = 10$, có 3 đồ vật:

- Vật 1: $w_1 = 5, v_1 = 2$
- Vật 2: $w_2 = 4, v_2 = 3$
- Vật 3: $w_3 = 7, v_3 = 4$

Tổng giá trị tối đa: $V_{\max} = 2 + 3 + 4 = 9$.

Khởi tạo: $dp[0] = 0$, các ô khác $= \infty$.

```

v:      0    1    2    3    4    5    6    7    8    9
-----
Init:    0   INF  INF  INF  INF  INF  INF  INF  INF  INF

Vật 1 (w=5, v=2):
dp:      0   INF   5   INF  INF  INF  INF  INF  INF  INF
(Đặt v=2 tốn w=5)

Vật 2 (w=4, v=3):
- v=5: dp[5] = min(INF, dp[2] + 4) = 9
- v=3: dp[3] = min(INF, dp[0] + 4) = 4
dp:      0   INF   5   4   INF   9   INF  INF  INF  INF

Vật 3 (w=7, v=4):
- v=9: dp[9] = min(INF, dp[5] + 7) = 16 (vượt quá W=10)
- v=7: dp[7] = min(INF, dp[3] + 7) = 11 (vượt quá W=10)
- v=6: dp[6] = min(INF, dp[2] + 7) = 12 (vượt quá W=10)
- v=4: dp[4] = min(INF, dp[0] + 7) = 7
dp:      0   INF   5   4   7   9   INF  INF  INF  INF

```

Quét kết quả:

Ta duyệt từ $v = 9$ lùi về 0:

- $v = 9$: $dp[9] = 16 > 10$ (Không thỏa mãn)
- $v = 8 \dots 6$: $dp > 10$ hoặc ∞
- $v = 5$: $dp[5] = 9 \leq 10$ (Thỏa mãn!)

$\implies$ **Đáp án tối ưu là $v = 5$** (Chọn vật 1 và vật 2: trọng lượng $5 + 4 = 9 \leq 10$).

5. Bảng So Sánh Chiến Lược Trước & Sau Đảo Nhấn

Tiêu chí	DP Thuận (Standard DP)	DP Đảo Nhấn (State Inversion DP)
Bản chất câu hỏi	"Với tài nguyên R , tối đa hóa giá trị V ?"	"Để đạt giá trị V , cần tối thiểu tài nguyên R ?"
Chỉ số mảng	Tài nguyên R (Trọng lượng, thời gian, chi phí)	Giá trị V (Lợi ích, số lượng, độ dài)
Giá trị ô nhớ	$\max V$	$\min R$
Khi nào áp dụng?	Khi miền giá trị của tài nguyên R nhỏ ($\leq 10^5$)	Khi tài nguyên R rất lớn ($\geq 10^9$), nhưng V nhỏ ($\leq 10^5$)
Trường hợp kinh điển	Knapsack 1 ($W \leq 10^5$)	Knapsack 2 ($W \leq 10^9, \sum v_i \leq 10^5$), LIS $O(N \log N)$
Độ phức tạp	$O(N \cdot R)$	$O(N \cdot \sum V)$

6. Cạm Bẫy Phòng Thi & Kinh Nghiệm Tối Ưu

1. **Khởi tạo giá trị vô cùng (`INF`):**
2. Không được dùng `0x3f3f3f3f` kiểu `int` nếu tổng trọng lượng có thể vượt quá 2×10^9 .
3. Dùng `const long long INF = 1e18;`.
4. Luôn kiểm tra điều kiện `dp[v - val] != INF` trước khi thực hiện phép cộng `dp[v - val] + weight` để tránh tràn số âm!
5. **Xác định đúng cận trên của mảng ($\sum V$):**
6. Đừng cố định kích thước mảng là hằng số quá lớn nếu số lượng phần tử nhỏ.
7. Tính tổng chính xác $\sum v_i$ của toàn bộ dữ liệu vào để chỉ duyệt v trong đoạn $[0, \sum v_i]$, tiết kiệm thời gian chạy đáng kể.
8. **Chiều duyệt khi nén mảng 1 chiều:**
9. Đối với bài toán Ba lô 0-1 (mỗi vật dùng tối đa 1 lần): Bắt buộc duyệt **ngược chiều** (v từ $\sum v$ giảm dần về v_i).
10. Đối với bài toán Ba lô không giới hạn (Unbounded Knapsack): Duyệt **xuôi chiều** (v từ v_i tăng dần lên $\sum v$).

7. Tài liệu Tham khảo & Đọc thêm

1. **AtCoder Educational DP Contest:**
2. [Task E - Knapsack 2](#) — Bài tập kinh điển chuẩn mực cho kỹ thuật đảo nhãn.
3. **CP-Algorithms:**
4. [Longest Increasing Subsequence](#) — Phân tích chi tiết chuyển đổi LIS từ $O(N^2)$ sang $O(N \log N)$ qua đảo nhãn.
5. **Codeforces Educational Blogs:**
6. [DP State Inversion and Dual Optimization](#) — Tổng hợp các dạng bài đảo nhãn trên Codeforces.
7. **VNOI Wiki:**
8. [Quy hoạch động cơ bản: Bài toán Ba lô](#) — Giới thiệu các biến thể của bài toán cái túi.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.